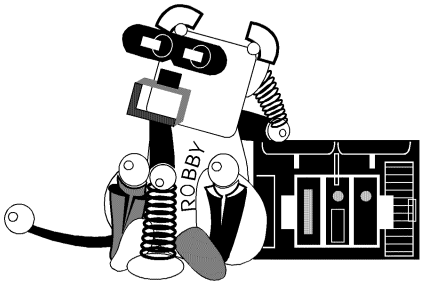
Глава III

ИГРА В СХЕМЫ



1. ПРАВИЛА ИГРЫ

Общие определения

Всякая схема имеет несколько (быть может, ни одного) входов и несколько (не менее одного) выходов. На каждом входе может быть один из двух сигналов: нуль или единица. Сигнал на каждом выходе определяется сигналами на входе (является функцией от сигналов на входе) и также может принимать значение, равное нулю или единице.

Мы будем изучать схемы, образованные некоторым образом из стандартных схем, описанных ниже.

Стандартные схемы

Схема `and` («И») имеет два входа и один выход. Сигнал на выходе равен единице, если оба входных сигнала равны единице, и нулю во всех остальных случаях.

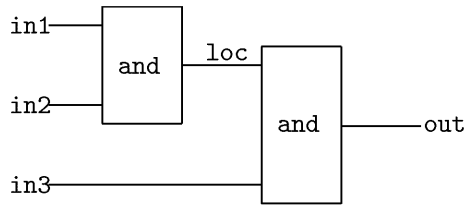
Схема `or` («ИЛИ») также имеет два входа и один выход, но сигнал на выходе равен единице, если хотя бы один из входных сигналов равен единице.

Схема отрицания `not` («НЕ») имеет один вход и один выход. Сигнал на ее выходе противоположен сигналу на входе.

Наконец, схемы `zero` («НУЛЬ») и `one` («ЕДИНИЦА») вовсе не имеют входов. Сигнал на их выходе постоянен и равен нулю и единице соответственно.

Правила образования новых схем

Мы продемонстрируем правила образования новых схем на примере. Соберем схему с тремя входами, на выходе которой будет единица, только если на всех трех входах будет единица. Графически ее можно изобразить в виде



Выход левой схемы `loc` является «локальным» (не выходит наружу построенной схемы, которую мы назовем `and3`).

Мы будем задавать схемы не рисунками, а описаниями. Изображенная выше схема описывается так:

```
scheme (in1 in2 in3) and3 (out):
local loc
  (in1 in2) and (loc)
  (loc in3) and (out)
end
```

Расположение слов по строкам значения не имеет, и наши отступы — это лишь декоративная наглядность.

При этом локальный (как и выходной) контакт должен встречаться в правой части ровно один раз. (Если он встречается более одного раза, может произойти «короткое замыкание», если на один и тот же провод одна схема захочет подать 0, а другая 1. Если же он не встречается, значение на проводе будет не определено.)

Имена контактов и схем могут быть любыми последовательностями букв (русских и латинских) и цифр,

начинающимися с буквы.

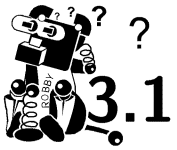
Вокруг списков входов и выходов обязательно должны стоять скобки. Порядок строк внутри описания роли не играет.

Обратите внимание на двоеточие, отделяющее заголовок от дальнейшего описания.

Описанную таким образом схему можно использовать наряду со стандартными.

При этом схема не должна содержать «обратной связи». Это означает, что ситуация, когда выход одной подсхемы использован как вход другой, ее выход — как вход третьей и т.п., а выход последней схемы — как вход первой, является недопустимой.

Кроме того, описания схем не должны содержать прямой или косвенной рекурсии.



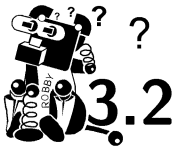
Определить схему «И» с четырьмя аргументами.

Решение:

```
scheme (in1 in2 in3 in4) and4 (out):
local loc
  (in1 in2 in3) and3 (loc)
  (loc in4) and (out)
end
```

◀

Замечание. В описаниях разных схем могут использоваться одни и те же имена контактов (это не играет роли). Но внутри одной схемы имена должны быть уникальны. В файле не может быть двух описаний схемы с одним и тем же именем. Имена стандартных схем также не могут быть переопределены.

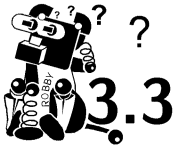


Привести другое решение предыдущей задачи, не использующее функции `and3`.

Решение:

```
scheme (in1 in2 in3 in4) and4 (out):
local loc1 loc2
  (in1 in2) and (loc1)
  (in3 in4) and (loc2)
  (loc1 loc2) and (out)
end
```
◄

2. ПОСТРОЕНИЕ ПРОСТЕЙШИХ СХЕМ



Составить схему, вычисляющую функцию «ИЛИ» трех аргументов (на выходе единица, если хотя бы на одном из входов единица).

Решение:

```
scheme (in1 in2 in3) or3 (out):
local temp
  (in1 in2) or (temp)
  (temp in3) or (out)
end
```
◄

Составить схему, вычисляющую функцию «ИЛИ» четырех аргументов. ◄

Составить схему с тремя входами, у которой выход равен единице или нулю в зависимости от того, каких входов больше (если есть хотя бы две единицы, то на выходе единица, если два нуля, то ноль). ◄

Составить схему `xor`[1]*, на выходе которой сумма двух входов по модулю 2 (на выходе единица, если ровно на одном входе единица).*

Решение:

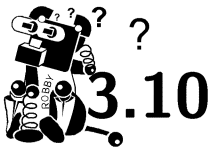
```
scheme (a b) xor (out):
local AnotB BnotA notA notB
  (a) not (notA)
  (b) not (notB)
  (a notB) and (AnotB)
  (b notA) and (BnotA)
  (AnotB BnotA) or (out)
end
```
◄

Построить схему с четырьмя входами, значение выхода которой изменяется при изменении значения любого ее входа. ◄

[1]От английского *exclusive or*

Составить схему с двумя входами и одним выходом, который равен единице только в том случае, когда значение первого входа меньше значения второго входа. ◄

Составить «элементарную сортирующую ячейку» с двумя входами и двумя выходами, которая пропускает сигналы со входов на выходы, если значение первого входа не меньше значения второго входа, и меняет их местами в противном случае. ◄



Составить схему с тремя входами `a`, `b` *и* `c` *и одним выходом, действующую согласно таблице:*

a	*0*	*0*	*0*	*0*	*1*	*1*	*1*	*1*
b	*0*	*0*	*1*	*1*	*0*	*0*	*1*	*1*
c	*0*	*1*	*0*	*1*	*0*	*1*	*0*	*1*
выход	*1*	*0*	*0*	*0*	*0*	*1*	*0*	*1*

Решение: Составим схему `notAnotBnotC`, равную единице только на нулевых входах, схему `AnotBC`, равную единице только на входах (1,0,1) и функцию `ABC`, равную единице только на (1,1,1). Далее применим операцию «ИЛИ» к трем этим функциям. ◄

3. НЕСКОЛЬКО ОБЩИХ УТВЕРЖДЕНИЙ

3.11 *Доказать, что любая функция из $\{0,1\}^n$ в $\{0,1\}$ может быть реализована схемой с n входами и одним выходом.*

Решение: Поступаем как в предыдущей задаче (для каждого единичного значения функции строим схему, равную единице только при соответствующих значениях входов, затем применяем «ИЛИ»). ◂

▹ Функция из $\{0,1\}^n$ в $\{0,1\}$ называется монотонной, если при увеличении любого из аргументов (с нуля до единицы) значение функции не уменьшается.

3.12 *Доказать, что любая монотонная функция может быть реализована схемой, не использующей стандартную схему* `not`. ◂

3.13 *Доказать, что стандартную схему* `not` *нельзя собрать из стандартных схем остальных четырех типов.*

Решение: Все остальные стандартные схемы — и, следовательно, любые их комбинации — монотонны. ◂

3.14 *Доказать, что любая функция может быть реализована схемой, составленной только из элементов* `and` *и* `not`. ◄

4. ОПЕРАЦИИ НАД ЧИСЛАМИ

3.15 *Описать схему, осуществляющую сложение двух двухразрядных двоичных чисел (четыре входа — два для каждого слагаемого) и три выхода (числа на выходе могут быть больше, чем на входе).* ◄

3.16 *Та же задача для четырехразрядных чисел.* ◄

3.17 *Та же задача для восьмиразрядных чисел.* ◄

3.18 *Описать схему, осуществляющую сравнение по величине двух восьмиразрядных чисел (на выходе единица, если первое число больше второго).* ◄

3.19 ?

Описать схему, осуществляющую вычитание двух восьмиразрядных чисел. ◂

3.20 ?

Описать схему, осуществляющую умножение двух восьмиразрядных чисел. ◂

3.21 ?

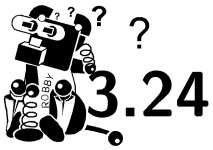
Описать схему, проверяющую, кратно ли трем восьмиразрядное двоичное число на ее входе. ◂

3.22 ?

Описать схему, которая делит записанное в двоичной системе число на максимальную степень двойки, на которую оно делится. ◂

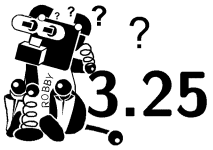
3.23 ?

Описать схему, осуществляющую деление восьмиразрядного числа на четырехразрядное (имеющую на выходе восемь разрядов для частного и четыре для остатка). ◂



Описать схему с семью входами, на выходе которой единица, если более трех входов имеют единицы. ◂

▹ Схема называется *сортирующей*, если число выходов равно числу входов, на выходах столько же единиц, сколько на входах и нули на выходах предшествуют единицам.



Составить сортирующую схему с восемью входами и выходами, используя в качестве элементов только сортирующие схемы с двумя входами и выходами. ◂